Source-Level Static Performance Estimation through Assembly-Supervised Learning

Lirong Yi
Chalmers University of Technology
Gothenburg, Sweden

***Abstract*—Estimating performance characteristics directly from source code can provide execution-related information early in software development. We present an assembly-supervised framework that predicts static instructions per cycle (IPC) from C functions. Source–assembly alignment supplies implementation information during training, while IPC regression organizes the source representation around labels generated by `llvm-mca` under a Skylake scheduling model. An instruction-aware (IA) tokenizer preserves whole opcodes and abstract operand classes within a compact assembly sequence. Prediction requires only the source encoder and a linear readout. On a fixed 2,000-function sample, a 7.52M-parameter compact encoder improves R^2 from 0.607 with IPC-only supervision to 0.752 with joint training. IA improves on BPE assembly encoding, which reaches 0.734. Evaluation on all 17,301 test functions extends the supervision benefit to CodeBERT and UniXcoder, with gains retained on 12,600 functions after filtering near-duplicates of training inputs. Every matched seed improves in R^2 for all three encoders on both expanded evaluation sets. Mixed-view training also supports prediction when declarations needed for compilation are missing, reaching $R^2 = 0.660$ on 1,136 declaration-stripped functions. Only 2.8% of these inputs pass compilation checks. These results establish paired assembly as useful training information for source-level static performance estimation across compact and pretrained representations.**

***Index Terms*—static performance prediction, instructions per cycle, source code representation, assembly supervision, contrastive learning**

I. Introduction

Estimating performance characteristics directly from source code can provide execution-related information early in software development. Developers inspect functions, compare implementations, and explore repositories at the source level. Associating these functions with execution-related estimates adds a numerical perspective to those activities and helps direct further analysis. Early prediction has been explored for worst-case execution time [1], functional-test execution time [2], and LLVM-based timing analysis [3].

Source-level analysis applies when a function is inspected separately from its surrounding declarations. A retrieved function or isolated excerpt may omit type definitions, macros, and forward declarations needed for compilation. Analysis of partial programs has a long history [4], and code snippets taken from Q&A sites are often not compilable [5]. Missing declarations can prevent compilation, leaving a compile-then-analyze pipeline without the assembly required by its analyzer. A source-level predictor can still produce an estimate directly from the available code, drawing on implementation information acquired during training. We study this setting through a predictor of static instructions per cycle (IPC), with labels supplied by `llvm-mca` under a fixed Skylake scheduling model.

The challenge is to connect the information expressed in source code with the behavior of its compiled implementation. Source exposes variables, expressions, and control structures; assembly exposes instructions and operands that use processor execution resources. A scalar IPC label identifies the numerical target, while the paired assembly reveals a structured implementation associated with that label. Their combination gives the source encoder both a target and an additional view from which to learn it.

The framework combines source–assembly contrastive alignment with source-side IPC regression. We study a compact Transformer trained from scratch and extensions to pretrained source encoders. An instruction-aware (IA) tokenizer represents assembly through whole opcodes and abstract operand classes, allowing more instructions to fit within the sequence budget. After training, a Ridge readout predicts IPC from frozen source embeddings. The assembly branch supplies training information; the prediction path retains the source encoder and a linear readout.

Controlled comparisons on 2,000 test functions show that the compact joint model reaches $R^2 = 0.752$, compared with 0.607 for IPC-only supervision and 0.488 for alignment alone. IA improves on BPE assembly encoding (0.734), while source-count gradient boosting reaches 0.585 with all training labels. We apply the same alignment principle to CodeBERT and UniXcoder under a shared 9,762-update budget.

On the full 17,301-function test partition, joint supervision improves every matched seed for all three source encoders. These gains persist after filtering near-duplicates of training functions, leaving 12,600 test inputs. This connects the compact-model ablations to a broader result: compiled implementations supply useful training information for both task-trained and pretrained source representations. The compact instantiation also provides a lightweight prediction path, with its encoder and readout taking 1.79 ms per pretokenized query on an A100.

We then extend the framework to functions that cannot be compiled in isolation because declaration context is missing. Mixing complete and declaration-stripped inputs during training yields $R^2 = 0.660$ on 1,136 declaration-stripped test functions while retaining 0.739 across all 2,000 complete test

inputs. Only 2.8% of the stripped functions pass compilation checks, yet the model produces predictions directly from their source. The source interface therefore makes static IPC estimates available when compilation cannot supply assembly to the analyzer.

The paper makes three contributions:

- An assembly-supervised framework for source-level static IPC prediction. Joint implementation alignment and numerical supervision strengthen source representations across compact and pretrained encoders.
- An instruction-aware assembly encoding that increases sequence coverage and improves the performance signal learned by the source encoder.
- An evaluation of supervision and assembly encoding across compact and pretrained source encoders, including full-partition prediction, training-overlap filtering, inference cost, and prediction with missing declaration context.

II. Background and Related Work

A. Source-Level Performance Prediction

Bonenfant et al. [1] investigated machine learning for early worst-case execution time prediction from C source, motivated by development stages where executable-level timing information is difficult to obtain. TEP-GNN [2] uses flow-augmented abstract syntax trees and graph neural networks to predict functional-test execution times in Java projects. More recently, GFTrans [6] combines source-derived control- and data-flow paths with explicit code features to classify C functions into four runtime categories. Our framework targets continuous static IPC prediction and uses compiled assembly as additional supervision for the source representation.

PrETi [3] predicts execution time using features extracted through LLVM-based execution-flow, cache, and branch simulation, together with historical timing data. Our framework uses compilation and `llvm-mca` during corpus construction, then transfers information from that analysis into a source encoder. Its static IPC target captures the instruction-stream behavior represented by the recorded processor configuration.

Böck et al. [24] evaluated source-based performance comparison across programming tasks and an industrial game-development domain, highlighting the importance of task and domain. We study whether paired assembly can strengthen that connection for a source-level static IPC predictor.

B. Instruction-Level Performance Models

Static instruction-stream models connect machine instructions to processor behavior. `llvm-mca` [25] uses LLVM scheduling information to estimate execution and resource behavior. OSACA analyzes assembly kernels, including critical paths and loop-carried dependencies [12]. uiCA models basic-block throughput on recent Intel microarchitectures [10], while Facile decomposes throughput prediction into analytical bottleneck models [11]. BHive [13] supplies hardware-measured basic-block throughput for validating such models, including `llvm-mca`, and DiffTune [14] learns `llvm-mca` parameters from hardware measurements.

Learned models capture this relationship from instruction-level training data. Ithemal [8] learns basic-block throughput from assembly using a recurrent model. GRANITE [9] represents basic blocks as graphs with structural and data dependencies and uses a graph neural network for throughput prediction. Concorde [20] combines analytical performance distributions with learned models for CPU performance prediction and architectural sensitivity analysis.

C. Performance-Oriented Representations

DeepTune [7] learns optimization heuristics from raw code for heterogeneous device mapping and GPU thread coarsening. IR2Vec [15] constructs embeddings from LLVM intermediate representations, while PERFOGRAPH [17] enriches IR graphs with numerical information and aggregate data structure representations for performance optimization and program analysis. ProGraML [16] represents programs as graphs over LLVM IR and evaluates them on data-flow analysis and compiler optimization tasks.

MIREncoder [18] combines IR-token and graph representations through multimodal pretraining, including IR–graph matching. Its reusable embeddings support several performance optimization tasks, including device mapping, thread coarsening, and loop vectorization. Our framework brings a related implementation-informed learning strategy to a source-input numerical predictor, using paired assembly and IPC labels to shape the source encoder.

PerfVec [19] learns program and microarchitecture representations for performance modeling, deriving program representations from instruction execution traces. We train source embeddings with paired assembly and a static numerical label, then evaluate their target information through a common Ridge readout.

D. Cross-Modal Code Learning

Asm2Vec [26] learns lexical and function-level assembly representations for binary clone search. PalmTree [27] learns instruction embeddings through assembly-oriented pretraining objectives, and Nova [28] combines hierarchical attention and contrastive objectives for assembly generation and understanding. LLM Compiler [23] pretrains language models on LLVM IR and assembly for compiler tasks. Our IA tokenizer exposes whole opcodes and operand categories to the assembly encoder.

Cross-modal supervision links low-level code to other views of a program. ContraBin [21] trains binary representations against anchored source and comment encoders. CLAP [22] aligns assembly with natural-language semantic explanations to learn transferable binary representations. Our framework directs the information toward the source branch: alignment with compiled implementations enriches a trainable source encoder, and IPC regression shapes its representation for numerical performance prediction. Using an additional view

TABLE I
POSITIONING OF RELATED APPROACHES BY PREDICTION INPUT AND OBJECTIVE.

Research line / examples	Prediction or downstream input	Prediction target / learning objective
Early WCET [1], TEP-GNN [2]	Source features / code structure	Worst-case execution time; functional-test execution time
GFTrans [6]	Source-derived flow paths and code features	Runtime-category prediction
DeepTune [7]	Source tokens	Heterogeneous device mapping and thread coarsening
PrETi [3]	LLVM simulation features	Early execution-time prediction
Ithemal [8], GRANITE [9]	Basic-block assembly	Learned basic-block throughput prediction
uiCA [10], Facile [11], OSACA [12]	Assembly blocks or kernels	Analytical throughput modeling; critical-path analysis
BHive [13], DiffTune [14]	Basic-block assembly with measured throughput	Hardware-measured throughput benchmark; learning `llvm-mca` parameters
IR2Vec [15], ProGraML [16], PERFOGRAPH [17], MIREncoder [18]	LLVM IR representations	Program representation learning for analysis and optimization
PerfVec [19], Concorde [20]	Execution-derived representations	Program / architecture representation learning; CPU performance modeling
ContraBin [21], CLAP [22]	Binary / assembly embeddings	Cross-modal representation learning for binary analysis
LLM Compiler [23]	LLVM IR and assembly	Pretrained language models for compiler tasks
PPA [4], CSnippex [5]	Incomplete programs / code snippets	Static analysis and compilable-code synthesis for partial code
This study	**Source code at inference**	**Static IPC prediction; assembly-supervised source representation learning**

during training also relates to learning using privileged information [29], where richer training information supports a decision rule operating on the primary input.

CodeBERT [30] and UniXcoder [31] provide general pretrained code encoders that can be adapted to IPC using source-only regression. Our experiments compare IPC-only adaptation with assembly-supervised adaptation of both pretrained encoders, alongside a compact encoder trained from scratch on the task corpus. Table I summarizes how these research lines differ in their prediction inputs and objectives.

III. METHOD

A. Overview

Each training example is a triplet (S_i, A_i, y_i) containing a C function, its compiled x86-64 assembly, and a static IPC label generated by `llvm-mca`. For a compilation and processor-analysis configuration c, we learn a source predictor of $y_c(S)$. IPC is the analyzed instruction count divided by the simulated cycle count, characterizing instruction execution under the selected processor scheduling model and analysis region.

Figure 1 illustrates the paired data with a maximum function, its compiled instructions, and its recorded static IPC of 2.96. The source and assembly encoders map their inputs into a shared 128-dimensional space. Contrastive learning associates each source function with its compiled instruction stream, and a linear IPC head encourages the source embedding to retain the numerical performance signal. After training, the frozen source encoder supplies embeddings to a separately fitted Ridge readout.

B. Instruction-Aware Tokenization

IA represents assembly as a sequence of opcode and operand-category tokens. Each instruction begins with a whole-opcode token. Register operands become typed classes, including general-purpose widths and SIMD classes; numeric immediates and memory displacements receive abstract markers. The encoder thus sees recurring patterns shared by instructions that differ in concrete register allocation or numeric values.

For example, `addq %rax, %rbx` becomes the token sequence `[ADDQ] [GPR64] [GPR64]`. The instruction `movq 8(%rax), %rbx` yields `[MOVQ] [MEM] [GPR64] [GPR64]`. Normalization preserves operand order. The tokenizer ignores assembler directives, standalone labels, and inline comments, and uses an unknown token for unrecognized opcodes. The resulting vocabulary contains 254 entries, including special tokens.

Whole-opcode tokens and abstract operands expose recurring instruction patterns while preserving register-width and vector-class distinctions. This representation covers longer instruction streams within the shared 512-token budget, supplying alignment with more of each paired implementation.

C. Encoders

The compact instantiation uses a 16,384-entry BPE source vocabulary. Both modalities have a maximum sequence length of 512, including boundary tokens, and longer inputs are truncated. Each compact encoder consists of token embeddings and learned positional embeddings followed by four pre-LayerNorm Transformer layers, with hidden dimension

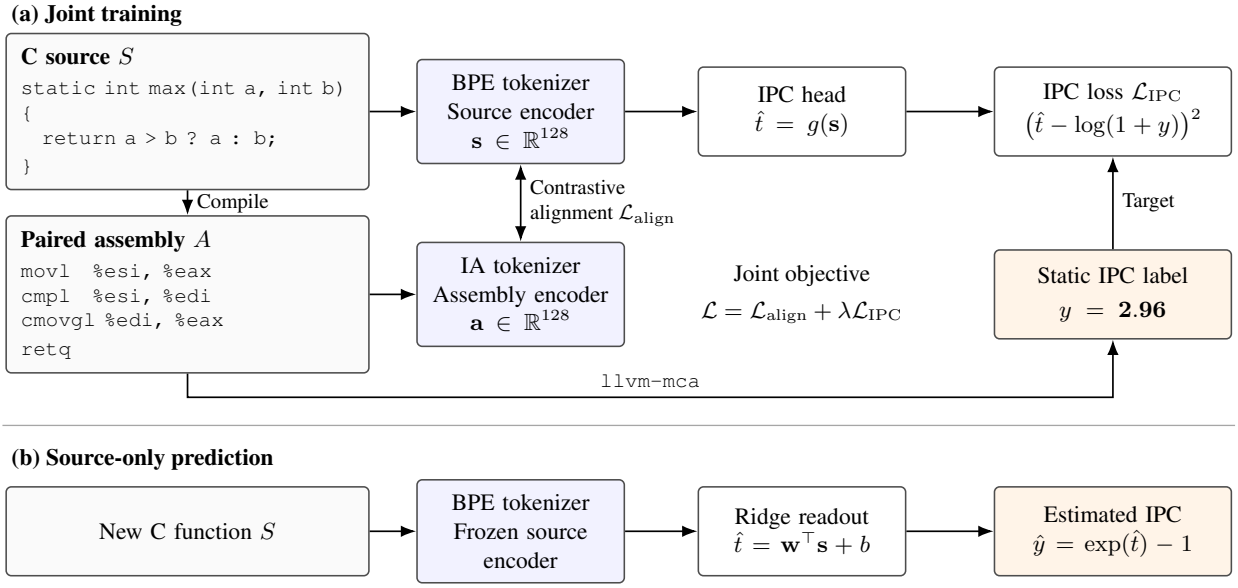


Fig. 1. Assembly-supervised source-level IPC estimation. (a) A training-corpus example links a C maximum function to its four compiled instructions and recorded static IPC label, $y = 2.96$. Source–assembly alignment and IPC regression jointly train the source representation. (b) A new function is encoded from source and mapped to an IPC estimate through the frozen encoder and Ridge readout. Source boilerplate, declaration attributes, and assembly directives are omitted from the example.

256, four attention heads, feed-forward dimension 1,024, and dropout 0.1. Masked mean pooling excludes padding positions; a final LayerNorm and bias-free linear projection produce the 128-dimensional embedding.

Writing the source and assembly embeddings as $\mathbf{s}_i = f_S(S_i)$ and $\mathbf{a}_i = f_A(A_i)$, the source branch contains 7,517,696 parameters. The joint IA training model, including the linear IPC head, contains approximately 10.91M parameters. Source-only prediction retains the source branch and a linear readout.

D. Joint Training

For a batch of B pairs, cosine similarity defines the cross-modal scores

$$q_{ij} = \frac{\langle \bar{\mathbf{s}}_i, \bar{\mathbf{a}}_j \rangle}{\tau}, \qquad \bar{\mathbf{s}}_i = \mathbf{s}_i / \|\mathbf{s}_i\|_2, \tag{1}$$

with analogous normalization for assembly and temperature $\tau = 0.05$. The positive match for source i is assembly i; other batch entries provide negatives. We minimize the mean of source-to-assembly and assembly-to-source cross-entropies, using uniform label smoothing of 0.05. This symmetric contrastive objective follows the paired-instance learning principle of InfoNCE [32].

A linear head g predicts the transformed target $t_i = \log(1 + y_i)$ from $\mathbf{s}_i$. The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{align}} + \lambda \frac{1}{B} \sum_{i=1}^{B} \big(g(\mathbf{s}_i) - t_i\big)^2. \tag{2}$$

The paired-instance objective asks the source representation to distinguish its compiled implementation from other implementations in the batch, providing supervision even when functions have similar IPC labels. Regression directs the representation toward distinctions relevant to the numerical target. The supervision ablation removes one term at a time to assess their combined benefit.

We calibrate the regression weight using training-label variance,

$$\lambda = \beta \frac{\mathrm{Var}(y)}{\mathrm{Var}(\log(1+y))}. \tag{3}$$

This expresses candidate regression weights on a comparable target-variance scale when selecting between raw and transformed labels. Validation selection yields $\beta = 0.6$ with the transformed target, corresponding to $\lambda = 5.9905$. IA and BPE use the same selected target and weight in the final comparison.

E. Source-Level Prediction

The final readout maps frozen source embeddings to transformed IPC. Given a fixed set $\mathcal{P}$ of labeled training examples, Ridge fits

$$\min_{\mathbf{w},b} \sum_{i \in \mathcal{P}} (\mathbf{w}^\top \mathbf{s}_i + b - t_i)^2 + \alpha \|\mathbf{w}\|_2^2, \tag{4}$$

with $\alpha = 1$ and an unpenalized intercept. No embedding standardization is applied. The output in IPC units is $\hat{y} = \exp(\mathbf{w}^\top \mathbf{s} + b) - 1$.

The regression head shapes the representation during joint training; the separately fitted Ridge readout turns that representation into the final predictor. Using the same readout across encoders also makes their performance information directly comparable.

F. Mixed-View Training

We construct a declaration-stripped input by retaining the function definition from the `__attribute__((used))` marker and removing its preceding context. The marker occurs in 1,136 of the 2,000 test functions. During joint training, each source is replaced by its declaration-stripped input with probability $p = 0.5$ when one exists. Functions without the marker remain unchanged. Assembly partners, IPC labels, architecture, optimizer, and update schedule retain the original configuration. This setting relates to partial-program analysis [4] and compilable-snippet synthesis [5], which recover missing context before analysis; our predictor instead estimates the target directly from the incomplete source.

We call the resulting model the *mixed-view model*; the original joint model uses complete inputs throughout training. Both predict the label of the original compiled implementation, so declaration-stripped inputs test prediction under controlled context removal. The primary Ridge readout uses complete inputs of the same 5,000 training functions; a secondary readout adds their declaration-stripped inputs as extra observations.

IV. Experimental Setup

A. Data

The corpus consists of AnghaBench-derived C functions [33], compiled into x86-64 assembly using an LLVM pipeline and paired with `llvm-mca` IPC labels. The original partitions contain 309,913 training functions, 17,150 validation functions, and 17,301 test functions. We retain these partitions and use the same record identities across all model comparisons. Re-running `llvm-mca` 18.1.8 with `-mcpu=skylake` and 100 iterations reproduced the recorded IPC labels to their stored precision on 2,346 functions with self-contained assembly from the full 17,301-function test partition. All labels refer to this fixed analysis configuration.

For encoder evaluation, a fixed sample of 5,000 training functions supplies the Ridge fitting set. Validation uses the first 2,000 records in the original validation partition. Controlled ablations, feature references, and declaration-removal experiments use the first 2,000 test records; expanded evaluation uses the full 17,301-function partition. These samples are fixed before the final model comparisons. Model initialization and training order vary across seeds, while the readout fitting set and evaluation functions remain unchanged.

B. Training

The compact encoders train from scratch for 9,762 parameter updates with batch size 256, including 256 paired examples in each contrastive loss. Optimization uses AdamW, learning rate 10^{-3}, weight decay 0.01, and gradient clipping at 0.5. The schedule has 200 warmup steps followed by cosine decay on a 10,000-step horizon, with a minimum multiplier of 0.1. The compact models use bfloat16 training on NVIDIA A100-SXM4-40GB GPUs. All encoder comparisons use 9,762 updates with effective batch size 256, corresponding to approximately 8.06 passes over the training set. Pretrained-model experiments use NVIDIA GH200 GPUs with PyTorch 2.9.1.

We select the IPC target and regression weight using validation probe R^2. Raw and log-transformed targets are evaluated with base weights 0.05, 0.15, 0.3, and 0.6 for each assembly representation. Candidates receive 2,000 updates; the two highest-validation candidates per representation continue to the full budget before final selection. The selected configuration is fixed for seeds 42, 43, and 44 and for the supervision ablations.

C. Baselines and Ablations

Pretrained source encoders. CodeBERT [30] and UniXcoder [31] are adapted under two conditions: IPC-only regression and joint assembly alignment with IPC regression. Each uses its native tokenizer, a maximum input length of 512, masked mean pooling over the complete 768-dimensional representation, and a linear IPC head. UniXcoder uses the encoder-only sequence format. Both conditions update the entire source encoder and head. The source encoders contain 124,055,040 and 125,339,136 parameters, respectively.

The joint condition adds the same IA assembly encoder used by the compact model and a bias-free source projection from 768 to 128 dimensions. The projection supplies the alignment embedding, while the IPC head operates on the full 768-dimensional source representation. Its objective is $\mathcal{L}_{\mathrm{MSE}} + \gamma \mathcal{L}_{\mathrm{align}}$, with the same temperature, label smoothing, and 256 paired examples as the compact joint model. The source architecture and IPC head remain identical between conditions. At prediction time, the assembly encoder and alignment projection are discarded; the common Ridge readout uses frozen 768-dimensional source embeddings.

Both pretrained conditions use bfloat16, 9,762 updates, effective batch size 256, and the compact model's AdamW decay, clipping, and step-based schedule. Microbatches contain 16 functions. A two-pass gradient cache preserves the full-batch objective by replaying encoder forward passes with identical dropout randomness.

Selection uses validation probe R^2 at seed 42. For each encoder, learning rates 10^{-5}, 2×10^{-5}, and 5×10^{-5} receive 2,000 IPC-only updates. The two leading candidates continue from their checkpoints to 9,762 updates. Both encoders select 5×10^{-5}, which is shared by their IPC-only and joint conditions. At that rate, joint-training alignment weights $\gamma \in \{0.03, 1/5.9905, 1.0\}$ follow the same screening and continuation procedure. CodeBERT selects 0.03 and UniXcoder selects $1/5.9905 \approx 0.16693$. Settings are frozen before test evaluation and shared across seeds 42, 43, and 44. Each seed starts from the original public pretrained checkpoint and uses the common Ridge and evaluation protocol.

Supervision ablations. The source-only IPC model retains our source encoder and regression objective and removes the assembly branch and alignment loss. The alignment-only model retains the source and IA assembly encoders and contrastive objective and removes the IPC training head. IPC labels enter the latter model only when fitting the post-training

Ridge readout. Both ablations use the same update budget and optimizer settings as the joint model; the source-only regression term retains the selected weight from Equation 2.

Assembly representation. The BPE variant replaces IA assembly tokenization with a 16,384-entry BPE vocabulary while retaining the source encoder, assembly Transformer architecture, sequence cap, and joint objective. This comparison measures the combined effect of assembly tokenization and its embedding vocabulary. Both variants use their validation-selected final configuration and three seeds.

Feature baselines. Hand-written counts feed gradient boosting (200 iterations, 15 or 31 leaves) and Ridge, with hyperparameters chosen on the validation split only. The source set has 11 counts: non-empty lines, characters, identifiers, numeric literals, loop keywords, branch keywords, call-like tokens, pointer stars, arithmetic operators, array accesses, and `return` statements, computed after removing comments and string literals. Assembly-input references use instruction count alone or counts and proportions of opcode categories, with AT&T operand parsing. Feature models use either the fixed 5,000-function sample or all 309,913 training functions. We also train an assembly-input Transformer with IA tokens, the same encoder structure, 9,762 updates, optimizer, and IPC objective. Its frozen embeddings use the same 5,000-function Ridge protocol. These references characterize prediction from the compiled view; the source-input models characterize prediction without assembly at query time.

D. Metrics

The primary metric is R^2 on $t = \log(1+\mathrm{IPC})$, matching the readout target. We additionally report R^2 and mean absolute error (MAE) in IPC units after inverse transformation. Neural-model results report the mean and sample standard deviation across seeds 42, 43, and 44. Feature baselines are single fits with validation-selected hyperparameters. Source embeddings are extracted in float32 for the readout evaluation. Model size refers to the source encoder retained at prediction time, including its projection where present; the final scalar prediction head is excluded from the encoder count.

E. Declaration Removal

Compilability is checked with `clang -fsyntax-only` (version 18.1.8) on the same 1,136 functions in complete and declaration-stripped form.

F. Expanded Evaluation

Expanded evaluation uses all 17,301 test functions. Pretrained Ridge readouts are reused; compact readouts are reconstructed from their original 5,000 fitting observations and regularization. All runs exactly reproduce the original 2,000-function predictions. Expanded-test observations enter neither selection nor fitting.

We assess training overlap using alpha-normalized lexical token 5-gram sets derived from complete source inputs. Normalization removes comments and whitespace, consistently renames identifiers by first occurrence, and abstracts numeric, string, and character literals. A test function is filtered when its set has Jaccard similarity at least 0.9 to any of the 309,913 encoder-training functions. Prefix filtering retrieves candidates, and explicit set intersections verify their similarity. This retains 12,600 of the full 17,301 test functions and 1,398 of the original 2,000. Thresholds 0.8 and 0.95 provide sensitivity checks. We report transformed-target R^2, IPC MAE, and Spearman rank correlation ρ with analyzed IPC on each evaluation set.

G. Inference Measurement

We measure all three source architectures sequentially on the same NVIDIA A100-SXM4-40GB GPU using PyTorch 2.5.1 and float32. Inputs are padded to the shared 512-token sequence length. Timing covers source encoding and a scalar linear readout, with 20 warmup iterations followed by 100 synchronized repetitions at batch sizes 1 and 16. We record mean latency, throughput, and peak GPU memory allocated by PyTorch; host tokenization is measured separately on 256 functions. GPU timings exclude tokenization and host–device transfers.

Timing uses the seed-42 compact joint checkpoint and archived IPC-adapted pretrained checkpoints with identical source architectures. Predictive evaluation uses the final checkpoints for every condition and seed.

V. Results

We first examine supervision and assembly encoding, then extend the comparison to pretrained source encoders. We evaluate prediction with missing declarations, full-partition performance under training-overlap filtering, and the cost of source-level inference.

A. Supervision and Assembly Encoding

Joint training substantially improves over either objective alone (Table II). Adding assembly alignment to the IPC-supervised source encoder raises transformed-target R^2 from 0.607 to 0.752, an absolute gain of 0.145. MAE decreases from 0.370 to 0.280, a 24.4% reduction. The complete model outperforms the source-only ablation at each of the three seeds.

Alignment alone produces a representation with $R^2 = 0.488$, showing that correspondence with compiled implementations carries information accessible to an IPC readout. Adding IPC supervision raises this score by 0.264 and reduces MAE by 40.9%. The numerical target is thus central to organizing the representation for prediction.

Figure 2 visualizes the prediction distributions under the three supervision configurations, using the same test functions, axes, and density scale.

Assembly instruction count alone reaches $R^2 = 0.140$ with gradient boosting (Table III). The learned source representation supports substantially more accurate prediction of the static target.

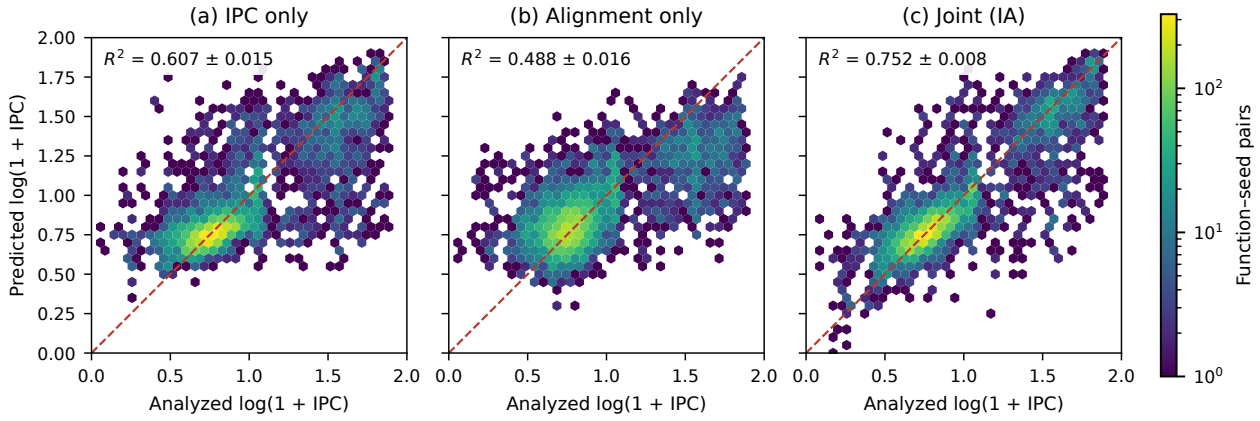


Fig. 2. Effect of supervision on source-level IPC prediction. Each panel shows the same 2,000 test functions under all three training seeds, giving 6,000 function–seed prediction pairs. The source architecture and 9,762-update budget are fixed. Hexagon counts use a common logarithmic color scale; the dashed line denotes exact prediction. Annotations report mean ± sample SD of individual-seed R^2. All predictions are included.

TABLE II
SUPERVISION AND ASSEMBLY REPRESENTATION COMPARISONS. ALL USE 9,762 UPDATES, THREE SEEDS, AND THE SAME RIDGE FITTING AND TEST SAMPLES.

Training configuration	$R^2_{\log(1+y)}$	MAE_{IPC}
IPC only	.607 ± .015	.370 ± .006
IA alignment only	.488 ± .016	.474 ± .011
BPE alignment + IPC	.734 ± .002	.293 ± .003
IA alignment + IPC	.752 ± .008	.280 ± .002

a) Assembly encoding.: IA improves transformed-target R^2 from 0.734 with BPE to 0.752, an absolute gain of 0.018. It also reduces MAE from 0.293 to 0.280, or 4.4%. IA has higher transformed-target R^2 at all three matched seeds. Since the source architecture and readout are unchanged, this comparison demonstrates that the assembly representation used during training affects the quality of source-only prediction.

The tokenization statistics illustrate how the representations allocate their sequence budget. Before truncation, training assembly has a mean length of 584.6 tokens under BPE and 107.4 under IA. With the common 512-position cap, 36.60% of training assembly sequences exceed the limit under BPE, compared with 2.35% under IA. IA therefore presents a more complete instruction stream within the same positional budget.

IA reduces the assembly vocabulary from 16,384 entries to 254, bringing the joint training model from approximately 15.04M to 10.91M parameters. Both models retain the same 7.52M-parameter source encoder for prediction. The assembly-side design therefore improves the learned predictor while preserving the source-side model size.

B. Supervision Across Encoders

We compare IPC-only and joint supervision within each source encoder. The compact Transformer and the two pretrained encoders span different model sizes and initialization strategies. Each comparison retains its source architecture and prediction readout, isolating the change in training supervision.

a) IPC-only adaptation.: Under the same 9,762-update budget, IPC-only fine-tuning gives CodeBERT $R^2 = .8597 \pm .0052$ and UniXcoder $R^2 = .8898 \pm .0044$, with IPC MAE of 0.195 and 0.177, respectively. The compact encoder is trained from scratch on the task corpus and uses approximately 6% of either pretrained encoder's parameters.

b) Assembly supervision beyond the compact encoder.: Adding assembly alignment raises CodeBERT's mean R^2 from 0.860 to $.8759 \pm .0069$ and reduces MAE from 0.195 to 0.188. All three matched seeds improve, with absolute R^2 gains of 0.008–0.023. Source architecture, learning rate, training examples, updates, and readout remain fixed, extending the supervision benefit to a broadly pretrained representation.

On the fixed 2,000-function sample, UniXcoder reaches $R^2 = .8912 \pm .0021$ with joint supervision and 0.890 with IPC-only adaptation. Its joint MAE is 0.180, compared with 0.177. The matched-seed R^2 changes are $+0.004$, $+0.006$, and -0.006. Section V-D evaluates these frozen models on the complete test partition and its training-overlap-filtered subset.

c) Feature and assembly references.: Table III places the source model against simple feature baselines on the same 2,000 test functions. Hand-written source counts with gradient boosting reach $R^2 = 0.475$ with the 5,000-function readout sample and 0.585 with all 309,913 training functions. The learned source model reaches 0.752 after encoder training on all 309,913 functions and Ridge fitting on 5,000 embeddings. Source counts with Ridge reach 0.067. These comparisons show the benefit of learned source representations over the evaluated structural counts.

Assembly category counts with gradient boosting reach $R^2 = 0.906$, and an assembly-input IA Transformer reaches 0.919 ± 0.003. These references connect the source-query task to the compiled view used for supervision.

TABLE III
FEATURE AND ASSEMBLY-INPUT REFERENCES. ALL USE THE SAME 2,000 TEST FUNCTIONS; R^2 IS ON $\log(1 + \text{IPC})$ AND MAE IS IN IPC UNITS. "5K" AND "ALL" DENOTE LABELED TRAINING SAMPLES. THE TRANSFORMER USES ALL TRAINING FUNCTIONS FOR ENCODER TRAINING AND 5K FOR RIDGE; ITS SCORES SUMMARIZE THREE SEEDS.

Prediction input / model	R^2	MAE
Source input		
Counts, Ridge (all)	.067	.634
Counts, GBM (5k)	.475	.431
Counts, GBM (all)	.585	.376
Assembly input		
Instruction count, GBM (all)	.140	.565
Category counts, GBM (all)	.906	.150
IA Transformer	.919 ± .003	.1313 ± .0005

TABLE IV
COMPLETE AND DECLARATION-STRIPPED INPUTS FOR THE SAME 1,136 TEST FUNCTIONS; THE FIRST COLUMN COVERS ALL 2,000 TEST FUNCTIONS. $R^2_{\log(1+y)}$, MEAN ± SAMPLE SD OVER THREE SEEDS, RIDGE FITTED ON COMPLETE INPUTS. MIXED-VIEW TRAINING USES $p = 0.5$. ONLY 2.8% OF DECLARATION-STRIPPED INPUTS PASS COMPILATION CHECKS.

Training	Complete (2,000)	Complete (1,136)	Stripped (1,136)
Original joint model	.752 ± .008	.704 ± .008	−.718 ± .071
Mixed-view model	.739 ± .003	.681 ± .002	.660 ± .012

C. Missing Declaration Context

The mixed-view model reaches $R^2 = 0.660 \pm 0.012$ on declaration-stripped inputs, with IPC MAE 0.317 (Table IV). On complete inputs for the same 1,136 functions it reaches 0.681 ± 0.002. Across all 2,000 complete test functions it retains $R^2 = 0.739 \pm 0.003$, compared with 0.752 for the original joint model. Prediction operates directly on source and remains available for stripped functions that fail compilation checks. Mixed-view training supports this use while largely preserving performance on complete inputs.

The transformation reduces mean source length from 323.6 to 190.8 BPE tokens and lowers the compilation-check pass rate from 100% to 2.8%. The original model, trained exclusively on complete inputs, obtains $R^2 = -0.718 \pm 0.071$ on declaration-stripped inputs. Exposure to both input forms during training restores prediction quality under this input change. Adding declaration-stripped inputs to the Ridge fitting observations yields mean $R^2 = 0.665$, close to the 0.660 obtained with complete-view readout fitting. The primary result therefore uses the same complete-view Ridge protocol as the other neural models.

D. Full Partition and Training Overlap

Table V extends the paired supervision comparison to all 17,301 test functions. Joint supervision raises R^2 from 0.889 to 0.900 for CodeBERT, from 0.909 to 0.914 for UniXcoder, and from 0.593 to 0.759 for the compact encoder. Each encoder also improves in mean IPC MAE and Spearman ρ. The larger evaluation therefore extends the supervision benefit to the pretrained encoders in both numerical error and target ordering.

Filtering removes 4,701 training-near-duplicates, leaving 12,600 functions. Of the full test partition, 1,212 functions have identical lexical token sequences to training inputs after comment and whitespace removal. On the retained subset, joint supervision raises R^2 from 0.828 to 0.843 for CodeBERT, from 0.858 to 0.866 for UniXcoder, and from 0.493 to 0.661 for the compact encoder. All three encoders improve at every matched seed on both the full partition and the filtered subset. The absolute gains on the filtered subset range from 0.012 to 0.021 for CodeBERT, 0.002 to 0.013 for UniXcoder, and 0.160 to 0.171 for the compact model.

IA also remains ahead of BPE when evaluation expands: compact joint R^2 increases from 0.748 to 0.759 on the full partition and from 0.645 to 0.661 after filtering. Spearman ρ improves from 0.787 to 0.797 and from 0.708 to 0.722, respectively. Across filtering thresholds 0.8, 0.9, and 0.95, the ordering of supervision conditions and the advantage of IA over BPE remain unchanged. Joint supervision improves each matched seed for all three encoders at every threshold. These results connect the controlled ablations to a larger evaluation with reduced training overlap.

E. Inference Cost

Table VI measures the retained prediction paths on one A100. The compact encoder and scalar readout take 1.79 ms per pretokenized query, compared with 8.98 ms for CodeBERT and 8.92 ms for UniXcoder. At batch size 16, the compact path processes 2,250 functions per second, compared with approximately 168 for either pretrained encoder. Peak allocated memory at batch size 1 is 48.7 MiB, compared with 500.4 MiB and 505.2 MiB. The compact instantiation thus provides approximately fivefold lower single-query latency and thirteenfold higher batched throughput under this measurement configuration.

Separately measured host tokenization averages 0.34 ms per function for the compact tokenizer, 0.83 ms for CodeBERT, and 0.95 ms for UniXcoder.

VI. DISCUSSION

A. A Source Interface to Static Analysis

The framework makes compilation-derived information available through a source query, supporting function inspection when surrounding build context is unavailable. Mixed-view training extends this interface to excerpts with missing declarations. Complete and stripped inputs retain the original implementation label, linking the available source to its analyzed static IPC.

B. What the Compiled View Contributes

Paired assembly associates source constructs with implementation structure beyond a scalar IPC label. IA exposes recurring instruction patterns and more of each implementation within the sequence budget. The supervision and tokenization

TABLE V

EXPANDED EVALUATION WITH FROZEN ENCODERS AND THE FIXED 5,000-FUNCTION RIDGE FITTING SET. THE FILTERED SUBSET EXCLUDES TRAINING-NEAR-DUPLICATES AT NORMALIZED TOKEN 5-GRAM JACCARD SIMILARITY ≥ 0.9. VALUES ARE MEAN $\pm$ SAMPLE SD OVER THREE SEEDS. R^2 USES $\log(1 + \mathrm{IPC})$, MAE IS IN IPC UNITS, AND ρ IS SPEARMAN CORRELATION.

	Full test partition (17,301)			Filtered subset (12,600)		
Encoder / supervision	R^2	MAE	ρ	R^2	MAE	ρ
CodeBERT / IPC	.889 ± .003	.157 ± .002	.876 ± .004	.828 ± .005	.178 ± .002	.816 ± .006
CodeBERT / joint	.900 ± .002	.150 ± .002	.884 ± .001	.843 ± .003	.170 ± .002	.8253 ± .0003
UniXcoder / IPC	.909 ± .003	.143 ± .001	.896 ± .002	.858 ± .005	.163 ± .001	.843 ± .003
UniXcoder / joint	.914 ± .001	.142 ± .001	.898 ± .001	.866 ± .001	.160 ± .002	.847 ± .002
Compact / IPC	.593 ± .009	.338 ± .005	.687 ± .006	.493 ± .008	.326 ± .005	.606 ± .004
Compact / joint (BPE)	.748 ± .003	.253 ± .001	.787 ± .002	.645 ± .004	.264 ± .001	.708 ± .003
Compact / joint (IA)	.7588 ± .0004	.244 ± .001	.797 ± .002	.661 ± .002	.2575 ± .0004	.722 ± .003

TABLE VI

SOURCE ENCODER PLUS SCALAR LINEAR READOUT ON THE SAME A100, IN FLOAT32 WITH 512-TOKEN PADDED INPUTS. LATENCY IS THE MEAN AT BATCH SIZE 1; THROUGHPUT USES BATCH SIZE 16. MEMORY IS PEAK PYTORCH-ALLOCATED GPU MEMORY AT BATCH SIZE 1. TOKENIZATION AND TRANSFERS ARE EXCLUDED.

Encoder	Latency (ms)	Throughput (fn/s)	Memory (MiB)
CodeBERT	8.98	168	500.4
UniXcoder	8.92	168	505.2
Compact	1.79	2250	48.7

comparisons show how this additional training view improves the source representation.

C. Supervision Across Representations

The paired-view principle benefits both task-trained and pretrained source encoders, with gains persisting after training-overlap filtering. Their different effect sizes show that assembly supervision can accompany different starting representations. The framework also supports different deployment costs: the compact instantiation provides lower encoding latency and memory use, while pretrained variants provide higher predictive accuracy.

VII. THREATS TO VALIDITY

The target is static IPC under one compilation and processor-analysis configuration. `llvm-mca` analyzes 100 iterations without following input-dependent control flow or executing callees. Hardware-measured throughput can differ from analyzer estimates [13], [14]; additional compiler and processor settings would test configuration transfer.

Training-overlap filtering evaluates sensitivity to lexical similarity; the retained subset also changes in composition. Project-disjoint evaluation would test transfer across code origins [24]. Three seeds measure training variation within these fixed data partitions. The ablations quantify objective contributions under the selected joint configuration, while feature references cover the specified counts and model families.

Mixed-view results concern one declaration-removal rule and mixing probability. An augmented IPC-only control would isolate assembly supervision's contribution to that adaptation. Pretrained models incorporate prior code pretraining beyond the shared task-training budget. Efficiency measurements concern float32 source encoding and linear prediction on one GPU, excluding tokenization and transfers.

VIII. CONCLUSION

Joint source–assembly alignment and IPC regression transfer implementation information into source-level static performance predictors. Controlled ablations show the value of both training objectives, while IA improves assembly coverage and the resulting source representation. Prediction retains only the source encoder and a linear readout.

On all 17,301 test functions and the 12,600-function training-overlap-filtered subset, joint supervision improves R^2 at every matched seed for the compact encoder, CodeBERT, and UniXcoder. The compact instantiation also provides a low-cost prediction path, taking 1.79 ms per pretokenized query on an A100. Mixed-view training extends source prediction to excerpts with missing declarations, reaching $R^2 = 0.660$ on 1,136 declaration-stripped functions. Paired compiled implementations therefore supply reusable training information for source-level static performance estimation, including inputs whose available context does not support independent compilation.

REFERENCES

[1] A. Bonenfant, D. Claraz, M. de Michiel, and P. Sotin, "Early WCET prediction using machine learning," in *17th International Workshop on Worst-Case Execution Time Analysis (WCET)*, ser. Open Access Series in Informatics, vol. 57, 2017, pp. 5:1–5:9.

[2] H. P. Samoaa, A. Longa, M. Mohamad, M. H. Chehreghani, and P. Leitner, "TEP-GNN: Accurate execution time prediction of functional tests using graph neural networks," in *Product-Focused Software Process Improvement (PROFES)*, ser. Lecture Notes in Computer Science, vol. 13709, 2022, pp. 464–479.

[3] R. Xu, P. Sieweck, H. von Hasseln, and D. Nowotka, "PrETi: Predicting execution time in early stage with LLVM and machine learning," *arXiv preprint arXiv:2503.13679*, 2025. [Online]. Available: https://arxiv.org/abs/2503.13679

[4] B. Dagenais and L. Hendren, "Enabling static analysis for partial Java programs," in *Proceedings of the 23rd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2008, pp. 313–328.

[5] V. Terragni, Y. Liu, and S.-C. Cheung, "CSnippex: Automated synthesis of compilable code snippets from Q&A sites," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, 2016, pp. 118–129.

[6] J. Li, Y. Wen, J. Liu, B. Zeng, and S. Mirjalili, "GFTrans: An on-the-fly static analysis framework for code performance profiling," *Frontiers in Big Data*, vol. 9, p. 1779935, 2026.

[7] C. Cummins, P. Petoumenos, Z. Wang, and H. Leather, "End-to-end deep learning of optimization heuristics," in *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2017, pp. 219–232.

[8] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin, "Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks," in *International Conference on machine learning*. PMLR, 2019, pp. 4505–4515.

[9] O. Sykora, P. M. Phothilimthana, C. Mendis, and A. Yazdanbakhsh, "GRANITE: A graph neural network model for basic block throughput estimation," in *2022 IEEE International Symposium on Workload Characterization (IISWC)*, 2022, pp. 14–26.

[10] A. Abel and J. Reineke, "uiCA: Accurate throughput prediction of basic blocks on recent Intel microarchitectures," in *Proceedings of the 36th ACM International Conference on Supercomputing*. ACM, 2022, pp. 1–14.

[11] A. Abel, S. Sharma, and J. Reineke, "Facile: Fast, accurate, and interpretable basic-block throughput prediction," in *2023 IEEE International Symposium on Workload Characterization (IISWC)*, 2023.

[12] J. Laukemann, J. Hammer, G. Hager, and G. Wellein, "Automatic throughput and critical path analysis of x86 and ARM assembly kernels," *arXiv preprint arXiv:1910.00214*, 2019. [Online]. Available: https://arxiv.org/abs/1910.00214

[13] Y. Chen, A. Brahmakshatriya, C. Mendis, A. Renda, E. Atkinson, O. Sykora, S. Amarasinghe, and M. Carbin, "BHive: A benchmark suite and measurement framework for validating x86-64 basic block performance models," in *2019 IEEE International Symposium on Workload Characterization (IISWC)*, 2019.

[14] A. Renda, Y. Chen, C. Mendis, and M. Carbin, "DiffTune: Optimizing CPU simulator parameters with learned differentiable surrogates," in *53rd IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020.

[15] S. VenkataKeerthy, R. Aggarwal, S. Jain, M. S. Desarkar, R. Upadrasta, and Y. N. Srikant, "IR2Vec: LLVM IR based scalable program embeddings," *ACM Transactions on Architecture and Code Optimization*, vol. 17, no. 4, pp. 1–27, 2020.

[16] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 2244–2253.

[17] A. TehraniJamsaz, Q. I. Mahmud, L. Chen, N. K. Ahmed, and A. Jannesari, "PERFOGRAPH: A numerical aware program graph representation for performance optimization and program analysis," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://papers.nips.cc/paper_files/paper/2023/hash/b41907dd4df5c60f86216b73fe0c7465-Abstract-Conference.html

[18] A. Dutta and A. Jannesari, "MIREncoder: Multi-modal IR-based pretrained embeddings for performance optimizations," in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2024. [Online]. Available: https://arxiv.org/abs/2407.02238

[19] L. Li, T. Flynn, and A. Hoisie, "Learning generalizable program and architecture representations for performance modeling," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–15.

[20] A. Nasr-Esfahany, M. Alizadeh, V. Lee, H. Alam, B. W. Coon, D. Culler, V. Dadu, M. Dixon, H. M. Levy, S. Pandey, P. Ranganathan, and A. Yazdanbakhsh, "Concorde: Fast and accurate CPU performance modeling with compositional analytical-ML fusion," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 1480–1494.

[21] Y. Zhang, C. Huang, Y. Zhang, H. Shao, K. Leach, and Y. Huang, "Pre-training representations of binary code using contrastive learning," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://openreview.net/forum?id=qmfUL6D0iz

[22] H. Wang, Z. Gao, C. Zhang, Z. Sha, M. Sun, Y. Zhou, W. Zhu, W. Sun, H. Qiu, and X. Xiao, "CLAP: Learning transferable binary code representations with natural language supervision," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 503–515.

[23] C. Cummins, V. Seeker, D. Grubisic, B. Roziere, J. Gehring, G. Synnaeve, and H. Leather, "Meta large language model compiler: Foundation models of compiler optimization," *arXiv preprint arXiv:2407.02524*, 2024.

[24] M. Böck, S. Habchi, M. Nayrolles, and J. Cito, "Performance prediction from source code is task and domain specific," in *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*, 2023, pp. 35–42.

[25] LLVM Project, "llvm-mca: LLVM machine code analyzer," https://llvm.org/docs/CommandGuide/llvm-mca.html, accessed: October 14, 2026.

[26] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 472–489.

[27] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an assembly language model for instruction embedding," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2021, pp. 3236–3251.

[28] N. Jiang, C. Wang, K. Liu, X. Xu, L. Tan, X. Zhang, and P. Babkin, "Nova: Generative language models for assembly code with hierarchical attention and contrastive learning," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=4ytRL3HJrq

[29] V. Vapnik and R. Izmailov, "Learning using privileged information: Similarity control and knowledge transfer," *Journal of Machine Learning Research*, vol. 16, no. 61, pp. 2023–2049, 2015. [Online]. Available: https://jmlr.org/papers/v16/vapnik15b.html

[30] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[31] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2022, pp. 7212–7225.

[32] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[33] A. F. Da Silva, B. C. Kind, J. W. de Souza Magalhães, J. N. Rocha, B. C. F. Guimaraes, and F. M. Q. Pereira, "Anghabench: A suite with one million compilable c benchmarks for code-size reduction," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 378–390.